

Revisão de Lógica: O Desafio dos Seis Pilares

Instruções Gerais:

Olá, Aventureiro(a) do Código! Esta lista de desafios foi criada para afiar suas habilidades nos seis pilares da nossa jornada até agora. Cada seção foca em uma ferramenta específica. Se sentir dificuldade, não hesite em consultar o "**Guia do Sábio**" no final deste documento. O objetivo é praticar e solidificar seu conhecimento. Vamos lá!

Seção 1: O Domínio das Variáveis e Tipos (5 Exercícios)

(Foco: Escolher o tipo certo de variável e manipular seus valores)

1. **Ficha de Monstro:** Crie variáveis para representar um monstro. Você precisará de uma para o nome (texto), uma para o hp (número inteiro), uma para a velocidadeDeAtaque (número com casas decimais), e uma para indicar se ele ehChefeDeFase (verdadeiro/falso). Atribua valores a elas e exiba uma ficha completa no console.
2. **Calculadora de Batalha:** Crie variáveis para o danoDoJogador (inteiro) e defesaDoMonstro (inteiro). Peça ao usuário para digitar esses valores. Calcule o danoFinal (dano - defesa). Exiba o resultado. Em seguida, crie uma variável float para a vidaRestantePercentual do monstro (assuma 100 de HP inicial) e calcule essa porcentagem.
3. **Conversor de Moedas Arcanas:** Um "Dragão de Ouro" vale 15.5 "Gemas de Prata". Crie um programa que pergunta ao jogador quantos "Dragões de Ouro" ele possui (pode ser um número inteiro). Calcule e exiba o valor equivalente em "Gemas de Prata" (que pode ser um número com casas decimais).
4. **Verificador de Condição:** Crie duas variáveis bool: jogadorTemChave e portaEstaAberta. Peça ao usuário para definir se são true ou false. Crie uma terceira variável bool chamada podePassar que será true somente se portaEstaAberta for true OU se jogadorTemChave for true. Exiba o valor de podePassar.
5. **Criador de Título de Herói:** Peça ao usuário seu nome (texto), sua classe (texto) e seu nível (inteiro). Crie uma única variável string chamada tituloCompleto que junte todas essas informações em uma frase, como: "Sir Kael, o Guerreiro de Nível 15". Exiba este título.

Seção 2: O Poder das Escolhas (if-else) (5 Exercícios)

(Foco: Criar lógicas de decisão complexas)

1. **O Baú do Tesouro com Armadilha:** Peça ao jogador o seu nível de "Ladinagem" (um inteiro). Se o nível for **maior que 15**, ele desarma a armadilha e ganha o tesouro. **Senão se** o nível for maior que 10 (mas não maior que 15), ele desarma a armadilha mas não ganha nada. **Senão**, a armadilha dispara e ele perde HP.
2. **Diálogo com Consequências:** Um guarda bloqueia seu caminho. Pergunte ao jogador como ele vai abordar o guarda: "persuadir", "subornar" ou "lutar".
 - Se "persuadir", role um dado (use Random). Se o resultado for alto (ex: > 10 num D20), ele consegue passar. Senão, o guarda fica irritado.
 - Se "subornar", pergunte quanto ouro ele oferece. Se for mais que 50, ele passa. Senão, o guarda recusa.
 - Se "lutar", o jogador perde HP, mas consegue passar.
3. **Sistema de Dano Elemental:** Peça ao jogador para escolher o tipo do seu ataque ("Fogo", "Água", "Grama") e o tipo do inimigo que ele está atacando. Use if-else aninhados para calcular o multiplicador de dano:
 - Fogo vs. Grama = Dano dobrado.
 - Água vs. Fogo = Dano dobrado.
 - Grama vs. Água = Dano dobrado.
 - Se os tipos forem iguais = Dano pela metade.
 - Em todos os outros casos = Dano normal.Exiba a mensagem: "É super eficaz!", "É pouco eficaz!" ou "Dano normal."
4. **Verificador de Acesso à Guilda:** Para entrar na Guilda dos Ladrões, um candidato precisa ter nível de "Furtividade" **maior que 12** E nível de "Agilidade" **maior que 14**, OU ter uma "Carta de Recomendação" (use uma variável bool). Crie um programa que peça esses três valores (dois int e um bool) e diga se o candidato foi aceito ou não.
5. **Clima e Viagem:** Pergunte ao jogador qual o clima atual ("Ensolarado", "Chuvoso", "Nevando") e que tipo de montaria ele está usando ("Cavalo", "Lobo", "Nenhuma"). Diga quanto tempo a viagem levará, aplicando penalidades:
 - Base de tempo: 10 horas.
 - Se "Chuvoso", adicione 2 horas. Se "Nevando", adicione 5 horas.
 - Se estiver de "Cavalo", subtraia 3 horas. Se de "Lobo", subtraia 4 horas.
 - **Desafio:** A penalidade de "Nevando" deve ser dobrada se o jogador estiver "Sem Montaria".

Seção 3: O Seletor de Opções (switch) (5 Exercícios)

(Foco: Usar switch para menus e estados definidos)

1. **Máquina de Poções:** Crie um menu que pergunta ao jogador qual poção ele quer: "[1] Cura", "[2] Força", "[3] Mana". Use um switch para a escolha. Cada case deve exibir uma mensagem sobre a poção escolhida e subtrair um custo diferente de uma variável de ouro.
2. **Teletransportador entre Cidades:** Peça ao jogador para digitar o nome da cidade para a qual ele quer viajar ("Ventobravo", "Orgrimmar", "Dalaran"). Use um switch no nome da cidade para dizer o custo da viagem e uma breve descrição do local.
3. **NPC com Personalidades Múltiplas:** Crie uma variável int chamada estadoEmocional que pode ser 1 (Feliz), 2 (Neutro), 3 (Triste) ou 4 (Irritado). Use um switch para que o NPC diga uma fala diferente para cada estado emocional.
4. **Aplicando Bônus de Arma:** Peça ao jogador para escolher um tipo de arma: "Espada", "Machado", "Arco". Use um switch para aplicar bônus diferentes a variáveis de dano e velocidadeDeAtaque.
 - "Espada": +5 de dano, +3 de velocidade.
 - "Machado": +8 de dano, -2 de velocidade.
 - "Arco": +4 de dano, +5 de velocidade.Exiba os atributos finais.
5. **Conversor de Runa para Elemento:** Peça ao jogador para digitar um símbolo de runa ('F', 'A', 'T'). Use um switch na runa (um char) para dizer a qual elemento ela corresponde: 'F' = Fogo, 'A' = Água, 'T' = Terra.

Seção 4: A Bolsa Dimensional (Arrays) (5 Exercícios)

(Foco: Criar, popular e manipular coleções de dados)

1. **Cinto de Utilidades:** Crie um array de string para representar um cinto com 4 espaços. Peça ao jogador para digitar o nome de 4 itens para preencher o cinto. Depois, exiba o conteúdo completo do cinto.
2. **Histórico de Batalha:** Crie um array de int para guardar o dano causado nos últimos 5 turnos. Peça ao jogador para digitar esses 5 valores de dano. Depois de preenchido, calcule e exiba o dano total e o dano médio causado nesses turnos.
3. **Procurando o Ingrediente:** Crie um array de string com 6 ingredientes para poções. Peça ao jogador para digitar o nome de um ingrediente que ele está procurando. Use um loop for e um if para varrer o array. Se encontrar o

ingrediente, exiba "Ingrediente encontrado no slot #[número do slot]!". Senão, exiba "Ingrediente não encontrado."

4. **Invertendo a Ordem dos Feitiços:** Crie um array de string com nomes de 4 feitiços. Use um loop for que comece do final do array e vá até o início para exibir a lista de feitiços em ordem inversa.
5. **Inventário de Itens Raros:** Crie dois arrays paralelos: um string[] nomesItens e um int[] valoresItens, ambos com tamanho 3. Preencha-os com itens e seus respectivos valores em ouro. Peça ao jogador para escolher um slot (0, 1 ou 2). Exiba o nome do item e seu valor correspondente.

Seção 5: Os Ciclos do Tempo (for e while) (5 Exercícios)

(Foco: Aplicar a estrutura de repetição correta para cada problema)

1. **Forjando a Espada (for):** Um ferreiro precisa de 8 marteladas para forjar uma espada. Use um loop for para simular as 8 marteladas, exibindo "Martelada #[número]..." a cada repetição. A cada 2 marteladas (use o operador %), exiba "O ferreiro mergulha a lâmina na água!".
2. **Escalando a Torre Infinita (while):** O jogador começa no andar 1 de uma torre. Use um loop while que continue enquanto o jogador digitar "subir". A cada volta, incremente o andar e exiba "Você chegou ao andar #[número]!". O loop para quando ele digitar qualquer outra coisa.
3. **Adivinhe o Código Secreto (while com limite):** O jogo tem um código secreto (ex: 1337). O jogador tem 3 tentativas para acertar. Use um loop while que continue enquanto o palpite estiver errado E o número de tentativas for menor que 3. Dê feedback se o jogador acertou ou se suas tentativas acabaram.
4. **Coletando Recursos (while):** O jogador precisa coletar 50 madeiras. A cada "turno", ele coleta uma quantidade aleatória de madeira (entre 5 e 10). Use um loop while que continue enquanto a quantidade de madeira coletada for menor que 50. Exiba a quantidade coletada a cada turno e o total.
5. **Batalha Final (while e for):** Simule uma batalha. O Chefe Final tem 100 de HP.
 - Use um loop while que continue enquanto o HP do Chefe for > 0.
 - Dentro do while, o jogador pode usar um "Combo de 3 Golpes". Use um loop for que rode 3 vezes para simular os 3 golpes.
 - Cada golpe causa um dano aleatório. Subtraia o dano do HP do Chefe e exiba o HP restante após cada combo.
 - No final, anuncie a vitória.

Guia do Sábio: Dicas e Explicações

Quando usar qual ferramenta?

- **Variáveis e Tipos:**
 - `string`: Para qualquer texto (nomes, frases, escolhas como "atacar").
 - `int`: Para números inteiros (HP, nível, contagens, quantidade de itens).
 - `float`: Para números com casas decimais (preços, porcentagens, dano preciso).
 - `bool`: Para responder perguntas de "sim/não" (`true/false`), como `temChave` ou `inimigoEstaAtordoad`.
 - `Random`: Para criar imprevisibilidade. Lembre-se de criar o objeto `Random` **UMA VEZ** e depois usar o método `.Next(min, max)` para gerar os números.
- **`if/else if/else`:**
 - **Quando usar?** Para tomar decisões baseadas em condições. É a ferramenta mais flexível para lógica.
 - **Dica:** Use `if` para uma única verificação. Use `if-else` para uma escolha de "dois caminhos". Use `if-else if-else` para testar uma sequência de múltiplas condições diferentes. Lembre-se dos operadores `&&` (E - ambos precisam ser verdadeiros) e `||` (OU - apenas um precisa ser verdadeiro).
- **`switch`:**
 - **Quando usar?** Quando você tem **UMA** única variável (geralmente um `int` ou `string`) e quer fazer algo diferente para **CADA VALOR ESPECÍFICO** que ela possa ter. É ótimo para menus.
 - **Dica:** É mais limpo que um `if-else if` longo quando todas as condições são `variavel == valor1, variavel == valor2, etc.` Não se esqueça do `break`; no final de cada `case`!
- **Arrays (`[]`):**
 - **Quando usar?** Sempre que você precisar guardar uma **coleção de itens do mesmo tipo**. Inventários, listas de inimigos, históricos de pontuação, etc.
 - **Dica:** Lembre-se que o primeiro item está no índice **0**. Para acessar o tamanho de um array, use `.Length`.
- **`for` loop:**
 - **Quando usar?** Quando você sabe **EXATAMENTE QUANTAS VEZES** quer repetir uma ação. "Faça isso 10 vezes", "Percorra todos os 5 itens deste array".
 - **Dica:** A estrutura `for (int i = 0; i < limite; i++)` é sua melhor amiga para trabalhar com arrays. `i` é o contador que você pode usar como índice.
- **`while` loop:**
 - **Quando usar?** Quando você quer repetir uma ação **ENQUANTO UMA CONDIÇÃO FOR VERDADEIRA**, mas você não sabe de antemão quantas vezes isso vai acontecer. "Continue a batalha enquanto o chefe tiver vida", "Mantenha o menu aberto até o jogador escolher sair".
 - **Dica de PERIGO:** Sempre garanta que algo **DENTRO** do loop possa, em algum momento, tornar a condição de parada `false`, para evitar um loop infinito

• EXEMPLOS

- **Variáveis e Tipos:**

- **O quê:** São "caixas" nomeadas para guardar informações. Cada caixa tem um tipo específico de item que pode guardar.
- **Quando usar:** Sempre que precisar que seu programa "lembre" de algo (nome do jogador, HP, se a chave foi encontrada, etc.).
- **Exemplo de Feitiço (Código):**

Generated csharp

```
// Uma variável de texto (string) para o nome
string nomeJogador = "Arion";
```

```
// Uma variável de número inteiro (int) para a vida
int hpAtual = 100;
```

```
// Uma variável de número com casas decimais (float) para o ouro
float ouro = 75.50f;
```

```
// Uma variável de verdadeiro/falso (bool) para um estado
bool temEscudo = true;
```

```
// Criando o "dado" para gerar números aleatórios
Random dado = new Random();
```

- **if / else if / else:**

- **O quê:** Uma estrutura para tomar decisões baseadas em condições. É como uma encruzilhada no seu código.
- **Quando usar:** Quando o fluxo do seu programa precisa mudar dependendo de uma ou mais condições serem verdadeiras ou falsas. É a ferramenta de decisão mais versátil.
- **Exemplo de Feitiço (Código):**

Generated csharp

```

    int nivelDoJogador = 12;
    int ouroDoJogador = 200;

    // SE o nível for alto E o ouro for suficiente
    if (nivelDoJogador > 10 && ouroDoJogador >= 150)
    {
        Console.WriteLine("Você pode comprar a Espada Lendária!");
    }
    // SENÃO SE, pelo menos, o nível for alto
    else if (nivelDoJogador > 10)
    {
        Console.WriteLine("Você tem o nível, mas não o ouro necessário.");
    }
    // SENÃO (se nenhuma das condições acima for verdadeira)
    else
    {
        Console.WriteLine("Volte quando for mais experiente e mais rico.");
    }

```

- **switch:**

- **O quê:** Um seletor que compara o valor de UMA variável com uma lista de valores específicos.
- **Quando usar:** Perfeito para menus, escolhas de classe, ou qualquer situação onde você tem uma variável e quer fazer algo diferente para cada valor exato que ela possa ter (case 1, case 2, case "atacar", etc.). É mais limpo que muitos if-else if para esses casos.
- **Exemplo de Feitiço (Código):**

Generated csharp

```

    Console.WriteLine("[1] Poção de Cura [2] Poção de Mana");
    int escolhaDoItem = int.Parse(Console.ReadLine());

    switch (escolhaDoItem)
    {
        case 1:
            Console.WriteLine("Você usou a Poção de Cura!");
            break; // Não se esqueça do break!
        case 2:

```

```

        Console.WriteLine("Você usou a Poção de Mana!");
        break;
    default:
        Console.WriteLine("Item não encontrado!");
        break;
}

```

- **Arrays ([]):**

- **O quê:** Uma "mochila" ou "lista" que guarda uma coleção de itens, todos do mesmo tipo.
- **Quando usar:** Para guardar inventários, listas de inimigos, nomes de feitiços, históricos de pontuação, etc. Qualquer coisa onde você tenha "vários" de "algo".
- **Exemplo de Feitiço (Código):**

Generated csharp

```

// Criando um array de string com 3 espaços e preenchendo-o
string[] inventario = new string[3];
inventario[0] = "Espada";    // O primeiro item está no índice 0
inventario[1] = "Poção";    // O segundo item está no índice 1
inventario[2] = "Mapa";     // O terceiro item está no índice 2

```

```

// Acessando um item específico
Console.WriteLine($"Item equipado: {inventario[0]}");

```

```

// Outra forma de criar já com os itens
int[] ultimasPontuacoes = { 150, 220, 185 };

```

- **for loop:**

- **O quê:** Uma estrutura de repetição que executa um bloco de código um número **exato** e **conhecido** de vezes.
- **Quando usar:** Quando você sabe de antemão quantas repetições precisa. Exemplos: "ataque 3 vezes", "conte por 5 segundos", "percorra TODOS os 10 itens do inventário". É a melhor ferramenta para trabalhar com arrays.
- **Exemplo de Feitiço (Código):**

Generated csharp


```
string[] inimigos = { "Goblin", "Orc", "Troll" };

// Este loop vai rodar 3 vezes (para i=0, i=1, i=2)
Console.WriteLine("Inimigos na área:");
for (int i = 0; i < inimigos.Length; i++)
{
    // 'i' é o nosso contador, que usamos como índice para o array
    Console.WriteLine($"{inimigos[i]}");
}
```

- **while loop:**

- **O quê:** Uma estrutura de repetição que executa um bloco de código **ENQUANTO** uma condição for verdadeira.
- **Quando usar:** Quando a repetição depende de um estado do jogo que pode mudar, e você não sabe quantas voltas vai levar. Exemplos: "continue a batalha enquanto o chefe tiver vida", "mantenha o jogo rodando até o jogador digitar 'sair'".
- **Dica de PERIGO:** Sempre garanta que algo **DENTRO** do loop possa, em algum momento, tornar a condição de parada false, para evitar um loop infinito!
- **Exemplo de Feitiço (Código):**

Generated csharp

```
int hpInimigo = 30;
int danoPorAtaque = 8;
int turno = 1;

// O loop continua enquanto o HP do inimigo for maior que 0
while (hpInimigo > 0)
{
    Console.WriteLine($"Turno {turno}: Você ataca!");
    hpInimigo = hpInimigo - danoPorAtaque; // A condição é alterada aqui!

    // Usamos Math.Max para não exibir HP negativo
    Console.WriteLine($"HP do inimigo restante: {Math.Max(0, hpInimigo)}");
    turno++;
}
```

```
Console.WriteLine("Inimigo derrotado!");
```